```
@@ -1,36 +0,0 @@
- use pyo3::prelude::*;
- use std::f64::consts::PI;
-
- const PHI: f64 = 1.618033988749895;
- const NODES: usize = 121;
-
- #[pyclass]
- struct SamaranKernel {
-     glyph_weights: Vec<f64>,
- }
-
- #[pymethods]
- impl SamaranKernel {
-     #[new]
-     fn new() -> Self {
-         let weights = (0..NODES).map(|i| {
-             (i as f64 * PHI).sin().abs()
-         }).collect();
-         SamaranKernel { glyph_weights: weights }
-     }
-
-     fn pre_determine(&self, input_signal: Vec<f64>) -> PyResult<f64> {
-         let mut resonance = 0.0;
-         for (i, val) in input_signal.iter().enumerate() {
-             let node_index = i % NODES;
-             resonance += val * self.glyph_weights[node_index] * PHI;
-         }
-         Ok(resonance / (input_signal.len() as f64))
-     }
- }
-
- #[pymodule]
```

```
- fn pre_d(_py: Python, m: &
PyModule) -> PyResult<()> {
-     m.add_class::<SamaranK
ernel>()?;
-     Ok(())
- }
```